Graph data structures in Go

A graph is a set of nodes (vertices) connected by edges. The data structure choice depends on edge density, query pattern, and update frequency. Here are the standard representations and the algorithms that map onto each.

Representations

Adjacency list

For each node, store the list of nodes it connects to.

```
type Graph struct {
    adj map[int][]int        // node -> neighbors
}

func NewGraph() *Graph {
    return &Graph{adj: map[int][]int{}}
}

func (g *Graph) AddEdge(u, v int) {
    g.adj[u] = append(g.adj[u], v)
    g.adj[v] = append(g.adj[v], u)         // omit this line for directed graph
}

func (g *Graph) Neighbors(u int) []int {
    return g.adj[u]
}
```

Space: O(V + E). Iterating neighbors is fast. Checking if a specific edge exists is O(degree).

Use when: - The graph is sparse (E is much less than V^2). - You traverse edges per node (BFS, DFS, most graph algorithms). - Nodes are added and removed dynamically.

Adjacency matrix

A 2D array where `m[u][v]` is 1 (or the weight) if there's an edge.

```go
type MatrixGraph struct {
    n   int
    mat [][]int       // mat[u][v] = weight, 0 = no edge
}

func NewMatrixGraph(n int) *MatrixGraph {
    mat := make([][]int, n)
    for i := range mat { mat[i] = make([]int, n) }
    return &MatrixGraph{n: n, mat: mat}
}

func (g *MatrixGraph) AddEdge(u, v, w int) {
    g.mat[u][v] = w
    g.mat[v][u] = w
}
```

Space: O(V^2) regardless of edge count. Edge existence check is O(1).

Use when: - The graph is dense (E approaches V^2). - You do frequent “is there an edge from u to v?” queries. - Algorithms benefit from matrix operations (Floyd-Warshall, transitive closure).

For V > a few thousand, the matrix dominates memory. Adjacency lists scale better.

Edge list

Just store every edge as a struct.

```go
type Edge struct { From, To, Weight int }

type EdgeListGraph struct {
    n     int
    edges []Edge
}
```

Space: O(E). Iterating all edges is fast; finding neighbors of a node is O(E).

Use when: - The algorithm processes edges in bulk (Kruskal’s MST, Bellman-Ford). - You don’t need neighbor queries. - Graph is loaded once and not updated.

Compressed sparse row (CSR)

Two flat slices: `head[i]` is the starting index in `edges` for node `i`’s out-edges; `head[i+1]` is one past the end.

```
type CSRGraph struct {
    head    []int       // size n+1
    edges   []int       // size E
    weights []int       // size E
}

// neighbors of u:
for i := g.head[u]; i < g.head[u+1]; i++ {
    nb := g.edges[i]
    w := g.weights[i]
    ...
}
```

Space: O(V + E) but with extreme cache locality. Used in high-performance graph engines and PageRank implementations.

Use when: - The graph is large and read-only after construction. - Performance matters (every edge traversal is a sequential read). - You don't add or remove edges after build.

Choosing the representation

	Adjacency list	Adjacency matrix	Edge list	CSR
Space	O(V+E)	$O(V^2)$	O(E)	O(V+E)
Neighbors of u	O(deg)	O(V)	O(E)	O(deg), cache-friendly
Edge exists?	O(deg)	O(1)	O(E)	O(deg)
Add edge	O(1)	O(1)	O(1)	hard (rebuild)
Remove edge	O(deg)	O(1)	O(E)	hard (rebuild)
Dense graph fit	poor	great	depends	poor
Sparse graph fit	great	wasteful	OK	great

Default: adjacency list with maps and slices. Switch to CSR for static, performance-critical graphs.

Directed vs undirected

The only structural difference: for an undirected edge u-v, you add both `u -> v` and `v -> u`. Most algorithms work the same; some (like topological sort, SCC) only make sense on directed graphs.

```
// undirected
g.adj[u] = append(g.adj[u], v)
g.adj[v] = append(g.adj[v], u)

// directed
g.adj[u] = append(g.adj[u], v)
```

Weighted vs unweighted

Add a weight to each edge. The simplest change: replace `[]int` with `[]Edge`:

```
type Edge struct { To int; Weight int }

type WGraph struct {
    adj map[int][]Edge
}

func (g *WGraph) AddEdge(u, v, w int) {
    g.adj[u] = append(g.adj[u], Edge{To: v, Weight: w})
}
```

Algorithms that need weights: Dijkstra, Bellman-Ford, Prim, Kruskal, A*. Algorithms that ignore weights: BFS, DFS, topological sort, cycle detection.

BFS (breadth-first search)

Visit nodes level by level. Use a queue.

```go
func BFS(g *Graph, start int) []int {
    visited := map[int]bool{start: true}
    var order []int
    queue := []int{start}
    for len(queue) > 0 {
        u := queue[0]
        queue = queue[1:]
        order = append(order, u)
        for _, v := range g.adj[u] {
            if !visited[v] {
                visited[v] = true
                queue = append(queue, v)
            }
        }
    }
    return order
}
```

Shortest path in unweighted graph

The BFS level when you first reach a node is the shortest path length (in edges).

```go
func ShortestPathUnweighted(g *Graph, start, end int) int {
    if start == end { return 0 }
    dist := map[int]int{start: 0}
    queue := []int{start}
    for len(queue) > 0 {
        u := queue[0]; queue = queue[1:]
        for _, v := range g.adj[u] {
            if _, seen := dist[v]; seen { continue }
            dist[v] = dist[u] + 1
            if v == end { return dist[v] }
            queue = append(queue, v)
        }
    }
    return -1
}
```

Common BFS uses

- Shortest path in unweighted graphs.
- Web crawling.
- Social network reach (“friends of friends”).
- Level-by-level analysis.
- Bipartite check.

DFS (depth-first search)

Visit deep before broad. Use a stack (or recursion).

```go
func DFS(g *Graph, start int) []int {
    var order []int
    visited := map[int]bool{}
    var dfs func(int)
    dfs = func(u int) {
        if visited[u] { return }
        visited[u] = true
        order = append(order, u)
        for _, v := range g.adj[u] {
            dfs(v)
        }
    }
    dfs(start)
    return order
}
```

For deep graphs, use iterative DFS to avoid stack overflow:

```go
func DFSIter(g *Graph, start int) []int {
    visited := map[int]bool{}
    var order []int
    stack := []int{start}
    for len(stack) > 0 {
        u := stack[len(stack)-1]
        stack = stack[:len(stack)-1]
        if visited[u] { continue }
        visited[u] = true
        order = append(order, u)
        for i := len(g.adj[u]) - 1; i >= 0; i-- {  // reverse for matching recursive
        order
            stack = append(stack, g.adj[u][i])
        }
    }
    return order
}
```

Common DFS uses

- Topological sort.
- Cycle detection.
- Strongly connected components (Tarjan, Kosaraju).
- Maze solving.
- Articulation points and bridges.

Cycle detection

Undirected (via DFS + parent tracking)

```go
func HasCycleUndirected(g *Graph, start int) bool {
    visited := map[int]bool{}
    var dfs func(u, parent int) bool
    dfs = func(u, parent int) bool {
        visited[u] = true
        for _, v := range g.adj[u] {
            if !visited[v] {
                if dfs(v, u) { return true }
            } else if v != parent {
                return true
            }
        }
        return false
    }
    return dfs(start, -1)
}
```

Directed (via DFS + recursion stack tracking)

```go
func HasCycleDirected(g *Graph) bool {
    visited := map[int]int{}        // 0 unvisited, 1 in stack, 2 done
    var dfs func(int) bool
    dfs = func(u int) bool {
        visited[u] = 1
        for _, v := range g.adj[u] {
            if visited[v] == 1 { return true }
            if visited[v] == 0 && dfs(v) { return true }
        }
        visited[u] = 2
        return false
    }
    for u := range g.adj {
        if visited[u] == 0 && dfs(u) { return true }
    }
    return false
}
```

The “in stack” state catches back-edges, which are cycles in a directed DFS tree.

Topological sort

Linearize a DAG so every edge `u -> v` has `u` before `v`. Two algorithms:

Kahn's algorithm (BFS-based)

```go
func TopologicalSort(g *Graph, n int) []int {
    indeg := make([]int, n)
    for _, neighbors := range g.adj {
        for _, v := range neighbors {
            indeg[v]++
        }
    }
    queue := []int{}
    for u := 0; u < n; u++ {
        if indeg[u] == 0 { queue = append(queue, u) }
    }
    var order []int
    for len(queue) > 0 {
        u := queue[0]; queue = queue[1:]
        order = append(order, u)
        for _, v := range g.adj[u] {
            indeg[v]--
            if indeg[v] == 0 { queue = append(queue, v) }
        }
    }
    if len(order) < n { return nil }          // cycle detected
    return order
}
```

DFS-based

Post-order DFS, then reverse.

```go
func TopoDFS(g *Graph, n int) []int {
    visited := make([]int, n)
    var order []int
    var dfs func(int) bool
    dfs = func(u int) bool {
        if visited[u] == 1 { return false }   // cycle
        if visited[u] == 2 { return true }
        visited[u] = 1
        for _, v := range g.adj[u] {
            if !dfs(v) { return false }
        }
        visited[u] = 2
        order = append(order, u)
        return true
    }
    for u := 0; u < n; u++ {
        if visited[u] == 0 && !dfs(u) { return nil }
    }
    // reverse order
    for i, j := 0, len(order)-1; i < j; i, j = i+1, j-1 {
        order[i], order[j] = order[j], order[i]
    }
    return order
}
```

Used for: build dependency orders, course prerequisite chains, dataflow analysis, package import ordering.

Dijkstra's algorithm

Shortest path from one source to all others in a non-negative weighted graph. Uses a priority queue.

See the heaps cheat sheet for the priority-queue implementation. Brief shape:

```go
func Dijkstra(g *WGraph, n, start int) []int {
    dist := make([]int, n)
    for i := range dist { dist[i] = math.MaxInt }
    dist[start] = 0
    pq := &PQ{}
    heap.Push(pq, &Item{Node: start, Dist: 0})

    for pq.Len() > 0 {
        u := heap.Pop(pq).(*Item)
        if u.Dist > dist[u.Node] { continue }       // stale
        for _, e := range g.adj[u.Node] {
            if alt := dist[u.Node] + e.Weight; alt < dist[e.To] {
                dist[e.To] = alt
                heap.Push(pq, &Item{Node: e.To, Dist: alt})
            }
        }
    }
    return dist
}
```

Complexity: O((V + E) log V) with a binary heap.

Fails on negative edges. For those, use Bellman-Ford.

Bellman-Ford

Single-source shortest paths, supports negative edges, detects negative cycles. Slower than Dijkstra: O(V*E).

```go
func BellmanFord(edges []Edge, n, start int) ([]int, bool) {
    dist := make([]int, n)
    for i := range dist { dist[i] = math.MaxInt }
    dist[start] = 0
    for i := 0; i < n-1; i++ {
        for _, e := range edges {
            if dist[e.From] != math.MaxInt && dist[e.From]+e.Weight < dist[e.To] {
                dist[e.To] = dist[e.From] + e.Weight
            }
        }
    }
    // check for negative cycle
    for _, e := range edges {
        if dist[e.From] != math.MaxInt && dist[e.From]+e.Weight < dist[e.To] {
            return nil, true   // negative cycle
        }
    }
    return dist, false
}
```

Floyd-Warshall

All-pairs shortest paths. $O(V^3)$. Dense graphs only.

```go
func FloydWarshall(n int, mat [][]int) [][]int {
    dist := make([][]int, n)
    for i := range dist {
        dist[i] = append([]int(nil), mat[i]...)
    }
    for k := 0; k < n; k++ {
        for i := 0; i < n; i++ {
            for j := 0; j < n; j++ {
                if dist[i][k] != math.MaxInt && dist[k][j] != math.MaxInt {
                    if d := dist[i][k] + dist[k][j]; d < dist[i][j] {
                        dist[i][j] = d
                    }
                }
            }
        }
    }
    return dist
}
```

Use when V is small (a few hundred) and you need every pair's distance.

Minimum spanning tree (MST)

Connect all nodes with minimum total edge weight, no cycles. Two classic algorithms:

Kruskal's

Sort edges by weight, take each one unless it creates a cycle. Uses union-find (see specialized DS sheet).

```go
func Kruskal(n int, edges []Edge) []Edge {
    sort.Slice(edges, func(i, j int) bool {
        return edges[i].Weight < edges[j].Weight
    })
    uf := NewUnionFind(n)
    var mst []Edge
    for _, e := range edges {
        if uf.Find(e.From) != uf.Find(e.To) {
            uf.Union(e.From, e.To)
            mst = append(mst, e)
            if len(mst) == n-1 { break }
        }
    }
    return mst
}
```

Prim's

Start from a node, grow the tree by repeatedly adding the cheapest edge connecting tree to non-tree. Uses a priority queue.

Both run in roughly O(E log V) time. Kruskal is cleaner when edges are pre-sorted or processed in bulk. Prim is better for dense graphs.

Strongly connected components (SCC)

In a directed graph, an SCC is a maximal set of nodes where every pair is mutually reachable. Two standard algorithms:

- **Kosaraju's:** Two DFS passes. First on the original, second on the reverse graph in reverse finish-time order.
- **Tarjan's:** One DFS pass with stack tracking and low-link values.

Both O(V + E). Use cases: dataflow analysis, dependency cycles, social network communities, circuit analysis.

A* search

Dijkstra with a heuristic. Prioritize nodes by `distance + heuristic_to_goal`. Faster when the heuristic is good (admissible: never overestimates).

```go
// inside the Dijkstra loop, change Priority:
priority := dist[v] + heuristic(v, goal)
heap.Push(pq, &Item{Node: v, Priority: priority})
```

Used in: pathfinding (game maps, robotics), planning, route-finding.

Graph libraries

The Go ecosystem has a few:

Library	Notes
`gonum.org/v1/gonum/graph`	Mature, generic, scientific computing focus
`github.com/dominikbraun/graph`	Generic, modern API
`github.com/yourbasic/graph`	Simple, no deps

For one-off algorithms in application code, write the small piece you need. For production graph algorithms (PageRank, community detection, max flow), reach for `gonum`.

Common patterns

Reverse a graph

```go
func Reverse(g *Graph) *Graph {
    out := NewGraph()
    for u, neighbors := range g.adj {
        for _, v := range neighbors {
            out.AddEdge(v, u)
        }
    }
    return out
}
```

Useful for: SCC, reachability from a target, etc.

Connected components

For undirected graphs, BFS/DFS from every unvisited node. Each pass labels one component.

```go
func ConnectedComponents(g *Graph, n int) [][]int {
    visited := make([]bool, n)
    var comps [][]int
    for u := 0; u < n; u++ {
        if visited[u] { continue }
        var comp []int
        queue := []int{u}
        visited[u] = true
        for len(queue) > 0 {
            x := queue[0]; queue = queue[1:]
            comp = append(comp, x)
            for _, y := range g.adj[x] {
                if !visited[y] {
                    visited[y] = true
                    queue = append(queue, y)
                }
            }
        }
        comps = append(comps, comp)
    }
    return comps
}
```

For directed graphs, “weakly connected” treats edges as undirected; “strongly connected” needs SCC.

Bipartite check

A graph is bipartite if you can 2-color it. Use BFS, color each node opposite to its parent. If you find an edge between same-colored nodes, not bipartite.

Performance considerations

Maps vs slices for adjacency

`map[int][]int` is flexible (handles non-dense node IDs). For dense IDs (0 to N-1), `[][]int` is faster:

```go
adj := make([][]int, n)
adj[u] = append(adj[u], v)
```

No hash, better cache locality. Use this when you control node IDs.

Memory: small graphs

A graph with 1 million nodes and 10 million edges as `[][]int`: maybe 200-400 MB depending on slice headers and growth slack. Fine on a server, painful on edge devices.

CSR for the same graph: closer to 80 MB. Worth the rebuild cost if the graph is static.

Algorithm choice for huge graphs

For graphs that don't fit in memory: - Stream processing on edge lists. - External-memory algorithms. - Distributed (Spark GraphX, Apache Giraph) - not Go territory.

For "large in memory": - Avoid the $O(V^2)$ matrix representation. - Use CSR if static. - Watch for slice growth overhead in adjacency lists; preallocate per-node capacity.

Best practices

1. **Use adjacency list with `[][]int` (or `[][]Edge`) for dense node IDs.** Faster than maps.
2. **Switch to CSR for static, read-heavy, performance-critical graphs.**
3. **For unweighted shortest path, use BFS** (not Dijkstra).
4. **Use Dijkstra with non-negative weights, Bellman-Ford otherwise.**
5. **Iterative DFS for very deep graphs.** Avoids stack overflow.
6. **Reuse visited slices via `sync.Pool`** in repeated BFS/DFS.
7. **Avoid `map[K]struct{}` for visited if K is dense.** A `[]bool` is faster.
8. **Mark nodes "in progress" vs "done" for cycle detection in DAGs.**
9. **For weighted graphs, store edges as structs**, not parallel arrays.
10. **Profile when graphs grow.** Adjacency list of `Edge` structs allocates per push; preallocate with `make([]Edge, 0, expectedDegree)`.

Common gotchas

Gotcha	Symptom	Fix
Undirected graph missing reverse edge	Half the connections invisible	Add both directions on insert
Directed cycle detection without “in stack” state	Misses cycles	Three-state visited (0/1/2)
Dijkstra on negative edges	Wrong results, no panic	Bellman-Ford
Adjacency list of 1B nodes	Memory blow-up	CSR or external storage
Matrix for sparse graph	$O(V^2)$ memory waste	Switch to adjacency list
BFS with `queue = queue[1:]`	Memory not reclaimed for long graphs	Use slice queue with compaction or ring buffer
Recursive DFS on million-deep graph	Stack overflow	Iterative DFS
`map[int]bool` visited on dense IDs	Slow, more allocations	`[]bool`
Topological sort returns partial result	Cycle in input	Check returned length == n
Forgetting to clear visited between BFS calls	Wrong results	Reset or use a generation counter
Floyd-Warshall on huge graph	$O(V^3)$ = death	Use Dijkstra per node, or just don’t
Storing weights as `int8` for big graphs	Overflow on long paths	Use `int` or check bounds
`int` for node IDs that should be uintptr-sized	Implicit assumption	Use `int` (platform-sized)
Mutating the graph during traversal	Hard to reason about	Snapshot or split read/write phases

Quick reference

Task	Algorithm	Complexity
Visit all nodes (any order)	BFS or DFS	O(V+E)
Shortest path, unweighted	BFS	O(V+E)
Shortest path, non-negative weighted	Dijkstra	O((V+E) log V)
Shortest path, with negative edges	Bellman-Ford	O(VE)
All-pairs shortest paths	Floyd-Warshall	$O(V^3)$
Pathfinding with heuristic	A*	O((V+E) log V) average
Cycle detection (undirected)	DFS + parent	O(V+E)
Cycle detection (directed)	DFS + state	O(V+E)
Topological sort	Kahn or DFS	O(V+E)
Connected components (undirected)	BFS/DFS per unvisited	O(V+E)
Strongly connected components	Tarjan or Kosaraju	O(V+E)
Minimum spanning tree	Kruskal or Prim	O(E log V)
Maximum flow	Ford-Fulkerson, Dinic's	varies
Bipartite check	BFS with 2-coloring	O(V+E)
Reachability	BFS/DFS from source	O(V+E)
Adjacency rep (sparse)	`[][]int` or `map[int][]int`	O(V+E)
Adjacency rep (dense)	`[][]int` matrix	$O(V^2)$
Static high-perf graph	CSR	O(V+E), cache-friendly